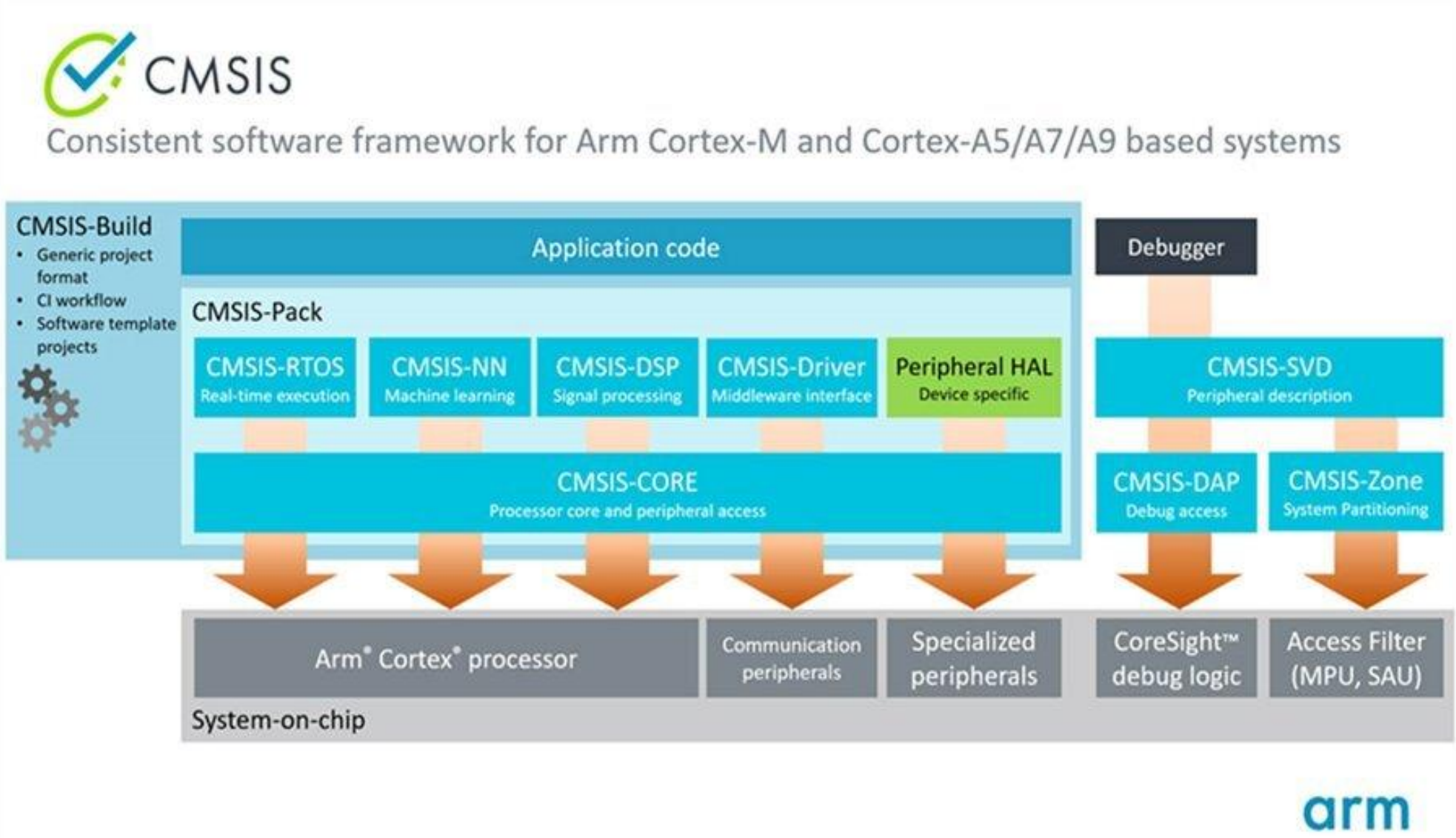
CMSIS-DSP Programming

CMSIS

- The Cortex Microcontroller Software Interface Standard (CMSIS)
  - is a vendor-independent hardware abstraction layer for microcontrollers that are based on Arm Cortex processors.
  - CMSIS defines generic tool interfaces and enables consistent device support.
    - CMSIS provides interfaces to processor and peripherals, real-time operating systems, and middleware components.
  - The CMSIS software interfaces simplify software reuse, reduce the learning curve for microcontroller developers, and improve time to market for new devices.
- Open source – License Apache 2.0
  - Current version 6.1
  - https://github.com/ARM-software/CMSIS_6/releases

CMSIS Components



CMSIS: support compiler

- Mainly ARM Compiler and GCC
  - ARM Compiler: current version 6
- Partial Support
  - IAR
- STM32CubeIDE uses GCC compiler
- Mbed Studio use ARMC6 compiler
- Mbed CLI uses ARMC6 or GCC

dotproduct_example_f32

- Compute the inner product of two 1-d array

```
float32_t srcA_buf_f32[MAX_BLOCKSIZE] =
{
  -0.4325648115282207,  -1.6655843782380970,  0.1253323064748307,
   0.2876764203585489,  -1.1464713506814637,  1.1909154656429988,
   1.1891642016521031,  -0.0376332765933176,  0.3272923614086541,
   0.1746391428209245,  -0.1867085776814394,  0.7257905482933027,
  -0.5883165430141887,   2.1831858181971011, -0.1363958830865957,
   0.1139313135208096,   1.0667682113591888,  0.0592814605236053,
  -0.0956484054836690,  -0.8323494636500225,  0.2944108163926404,
  -1.3361818579378040,   0.7143245518189522,  1.6235620644462707,
  -0.6917757017022868,   0.8579966728282626,  1.2540014216025324,
  -1.5937295764474768,  -1.4409644319010200,  0.5711476236581780,
  -0.3998855777153632,   0.6899973754643451
};

/* ------------------------------------------------------------------
** Test input data of srcB for blockSize 32
** ------------------------------------------------------------------ */
float32_t srcB_buf_f32[MAX_BLOCKSIZE] =
{
   1.7491401329284098,  0.1325982188803279,   0.3252281811989881,
  -0.7938091410349637,  0.3149236145048914,  -0.5272704888029532,
   0.9322666565031119,  1.1646643544607362,  -2.0456694357357357,
  -0.6443728590041911,  1.7410657940825480,   0.4867684246821860,
   1.0488288293660140,  1.4885752747099299,   1.2705014969484090,
  -1.8561241921210170,  2.1343209047321410,   1.4358467535865909,
  -0.9173023332875400, -1.1060770780029008,   0.8105708062681296,
   0.6985430696369063, -0.4015827425012831,   1.2687512030669628,
  -0.7836083053674872,  0.2132664971465569,   0.7878984786088954,
   0.8966819356782295, -0.1869172943544062,   1.0131816724341454,
   0.2484350696132857,  0.0596083377937976
};
```

- C[i] = A[i]*B[i]

```
arm_mult_f32(srcA_buf_f32, srcB_buf_f32, multOutput, MAX_BLOCKSIZE);
```

- Compute C[0]+C[1]+……+C[MAX_BLOCKSIZE]

```
for(i=0; i< MAX_BLOCKSIZE; i++)
{
  arm_add_f32(&testOutput, &multOutput[i], &testOutput, 1);
}
```

- Compare the result with that of matlab

```
/* absolute value of difference between ref and test */
diff = fabsf(refDotProdOut - testOutput);

/* Comparison of dot product value with reference */
status = (diff > DELTA) ? ARM_MATH_TEST_FAILURE : ARM_MATH_SUCCESS;
```

- Output

```
SUCCESS
----------------------
multOutput =
-0.756616
-0.220854
0.040762
-0.228360
-0.361051
-0.627935
1.108618
-0.043830
-0.669532
-0.112533
-0.325072
0.353292
-0.617043
3.249836
-0.173291
-0.211471
2.276826
0.085119
0.087739
0.920643
0.238641
-0.933381
-0.286860
2.059896
0.542081
0.182982
0.988026
-1.429069
0.269341
0.578676
-0.099346
0.041130
----------------------
testOutput = 5.927364
----------------------
```

CMSIS-DSP optimization

- loop-unrolling

```
while(blkCnt > 0u)
{
  /* C = A + B */
  /* Add and then store the results in the destination buffer. */

  /* read four inputs from sourceA and four inputs from sourceB */
  inA1 = *pSrcA;
  inB1 = *pSrcB;
  inA2 = *(pSrcA + 1);
  inB2 = *(pSrcB + 1);
  inA3 = *(pSrcA + 2);
  inB3 = *(pSrcB + 2);
  inA4 = *(pSrcA + 3);
  inB4 = *(pSrcB + 3);

  /* C = A + B */
  /* add and store result to destination */
  *pDst = inA1 + inB1;
  *(pDst + 1) = inA2 + inB2;
  *(pDst + 2) = inA3 + inB3;
  *(pDst + 3) = inA4 + inB4;

  /* update pointers to process next samples */
  pSrcA += 4u;
  pSrcB += 4u;
  pDst += 4u;


  /* Decrement the loop counter */
  blkCnt--;
}
```

```
void arm_add_f32 ( const float32_t *  pSrcA,
                   const float32_t *  pSrcB,
                   float32_t *        pDst,
                   uint32_t           blockSize
                 )
```

Parameters

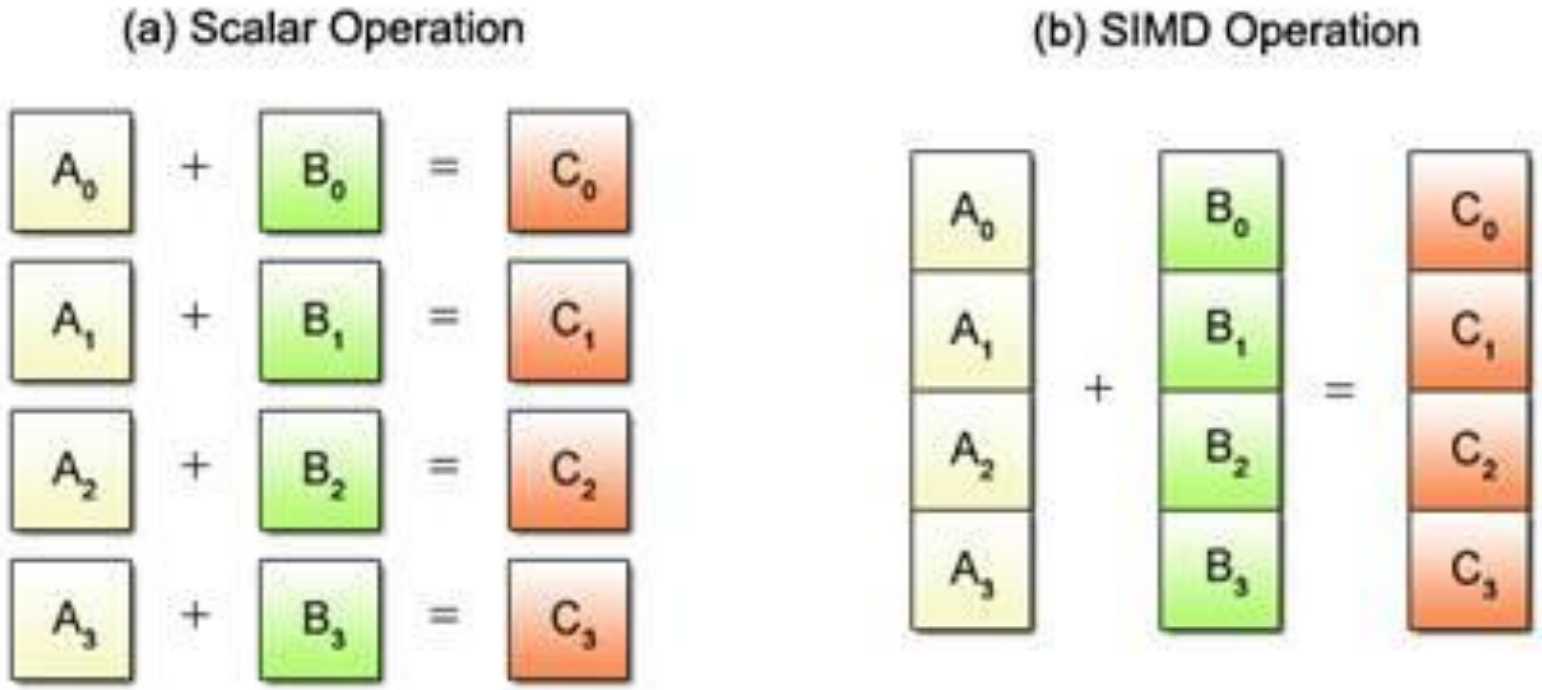
[in]	**pSrcA**	points to first input vector
[in]	**pSrcB**	points to second input vector
[out]	**pDst**	points to output vector
[in]	**blockSize**	number of samples in each vector

Returns

none

arm_add_f32() code snippet

Introduction to SIMD instructions



https://ftp.cvut.cz/kernel/people/geoff/cell/ps3-linux-docs/CellProgrammingTutorial/BasicsOfSIMDProgramming.html

https://github.com/ARM-software/CMSIS/blob/master/CMSIS/DSP_Lib/Source/BasicMathFunctions/arm_add_q15.c

- SIMD

```
while(blkCnt > 0u)
{
  /* C = A + B */
  /* Add and then store the results in the destination buffer. */
  inA1 = *__SIMD32(pSrcA)++;
  inA2 = *__SIMD32(pSrcA)++;
  inB1 = *__SIMD32(pSrcB)++;
  inB2 = *__SIMD32(pSrcB)++;

  *__SIMD32(pDst)++ = __QADD16(inA1, inB1);
  *__SIMD32(pDst)++ = __QADD16(inA2, inB2);

  /* Decrement the loop counter */
  blkCnt--;
}
```

arm_add_q15() code snippet

Note that inA1, inB1, inA2, inB2
are 32-bit entity (2 16-bit integers) .

__QADD16()
This instruction performs two signed integer additions on the corresponding halfwords of the operands and writes the results into the corresponding halfwords of the destination.

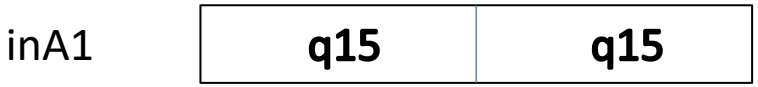
arm_add_q15() code implementation

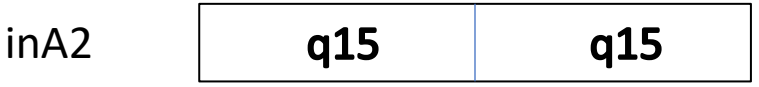
- pSrcA points to a vector of q15 fixed-point numbers
- pSrcB points to a vector of q15 fixed-point numbers

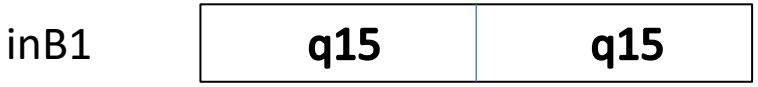
- inA1 = *__SIMD32(pSrcA)++
- inB1 = *__SIMD32(pSrcB)++

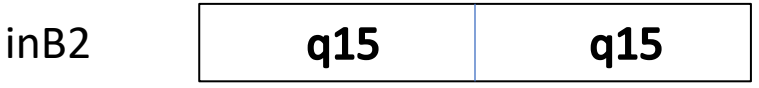
inA2 = *__SIMD32(pSrcA)++

inB2 = *__SIMD32(pSrcB)++









*__SIMD32(pDst)++ = __QADD16(inA1, inB1)

*__SIMD32(pDst)++ = __QADD16(inA2, inB2)

Review question:

- What is the **Range of Representable Values in** the ARM fixed point q15 (Q1.15) format?

(Hint: 1000 0000 0000 0000 ~ 0111 1111 1111 1111

-1 ~ 1-2^(-15)

Q format: please refer to https://en.wikipedia.org/wiki/Q_(number_formatLinks to an external site.)Links to an external site.

ARM q15 is a Q1.15 format fixed-point number.)

FIR example

- CMSIS-DSP github:
  - https://github.com/ARM-software/CMSIS-DSP
- Program examples
  - https://github.com/ARM-software/CMSIS-DSP/tree/main/Examples/ARM/arm_fir_example
- Documentation
  - https://arm-software.github.io/CMSIS_5/DSP/html/group__FIRLPF.html

FIR Lowpass Filter Example

- Removes high frequency signal components from the input using an FIR lowpass filter.
  - The example demonstrates how to configure an FIR filter and then pass data through it in a block-by-block fashion.



Input signal and lowpass filter design

- The input signal is a sum of two sine waves: 1 kHz and 15 kHz.
  - This is processed by an FIR lowpass filter with cutoff frequency 6 kHz.
  - The lowpass filter eliminates the 15 kHz signal leaving only the 1 kHz sine wave at the output.
- The lowpass filter was designed using MATLAB with a sample rate of 48 kHz and a length of 29 points. The MATLAB code to generate the filter coefficients is shown below:

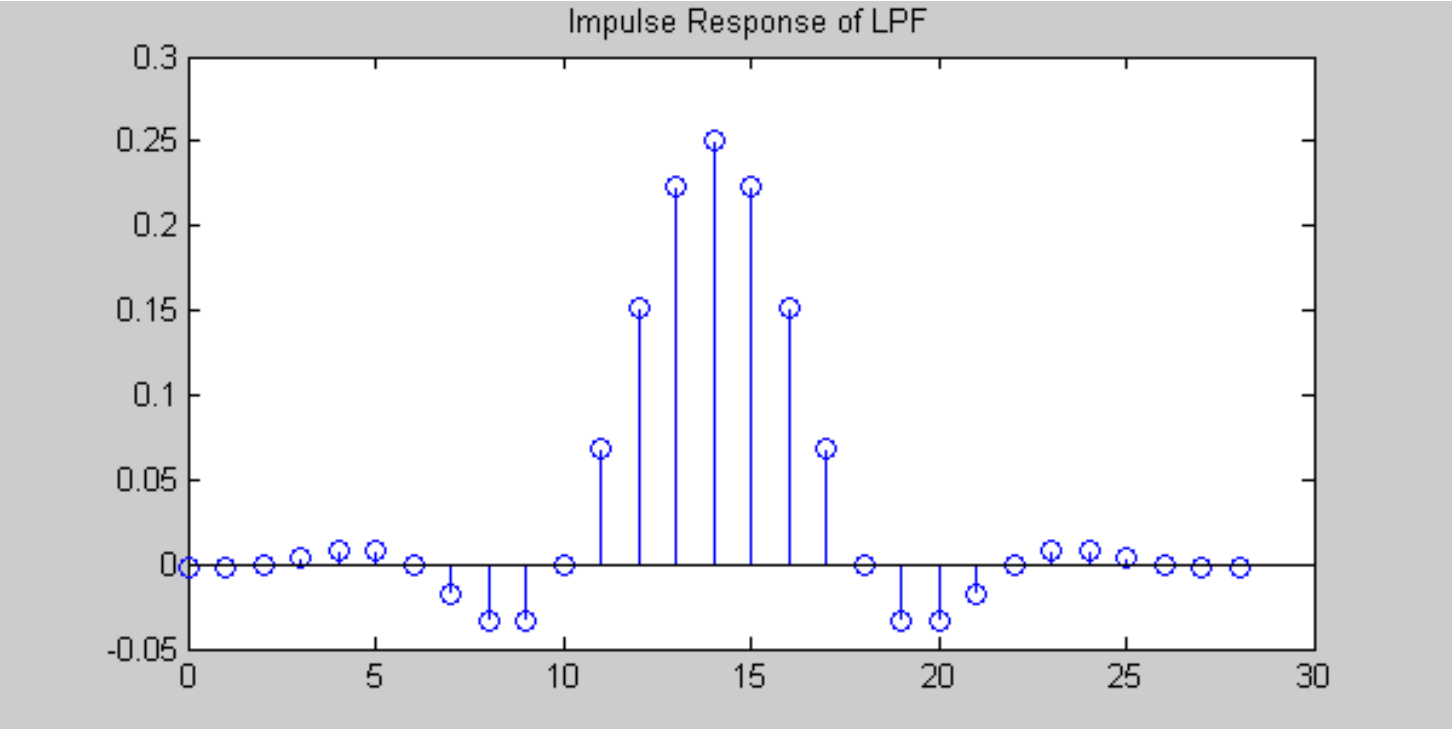
  h = fir1(28, 6/24);
  - The first argument is the "order" of the filter and is always one less than the desired length. The second argument is the normalized cutoff frequency.
  - A 6 kHz cutoff with a Nyquist frequency of 24 kHz lies at a normalized frequency of 6/24 = 0.25.
- The CMSIS FIR filter function requires the coefficients to be in time reversed order.

  fliplr(h)

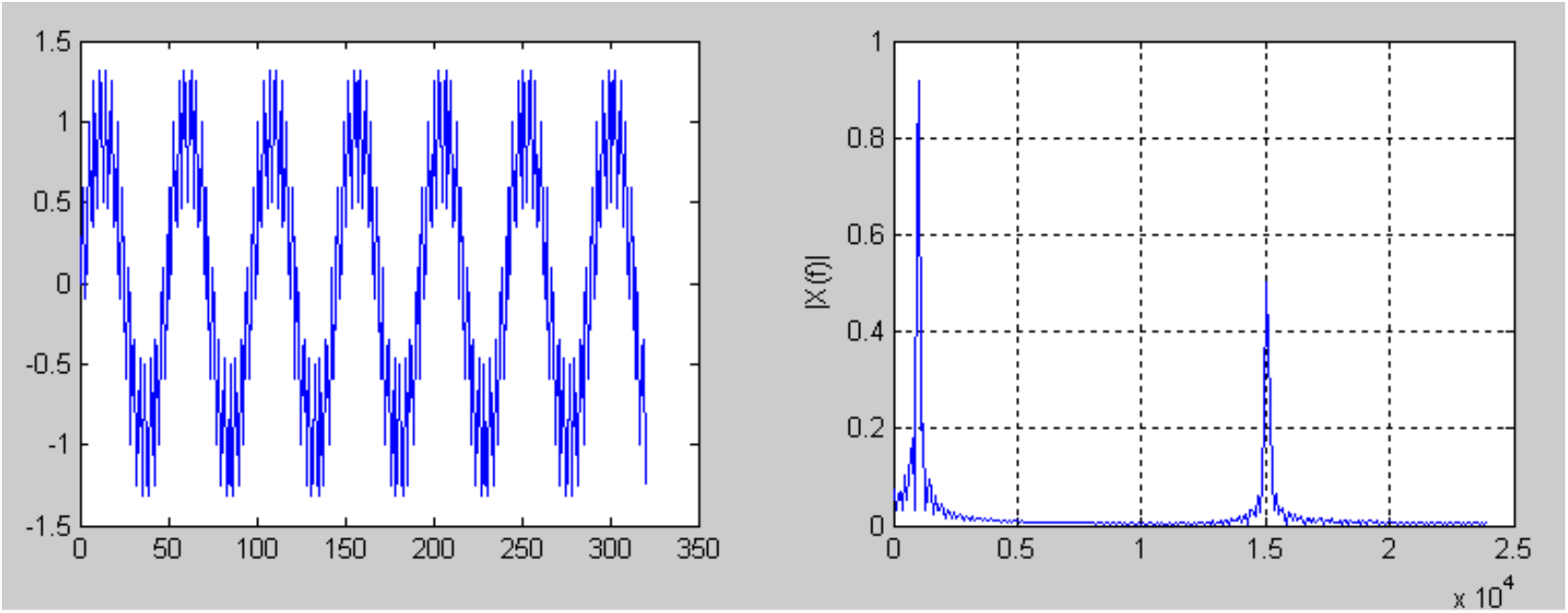
Note: B = fliplr(A) returns A with its columns flipped in the left-right direction.

The filter coefficients

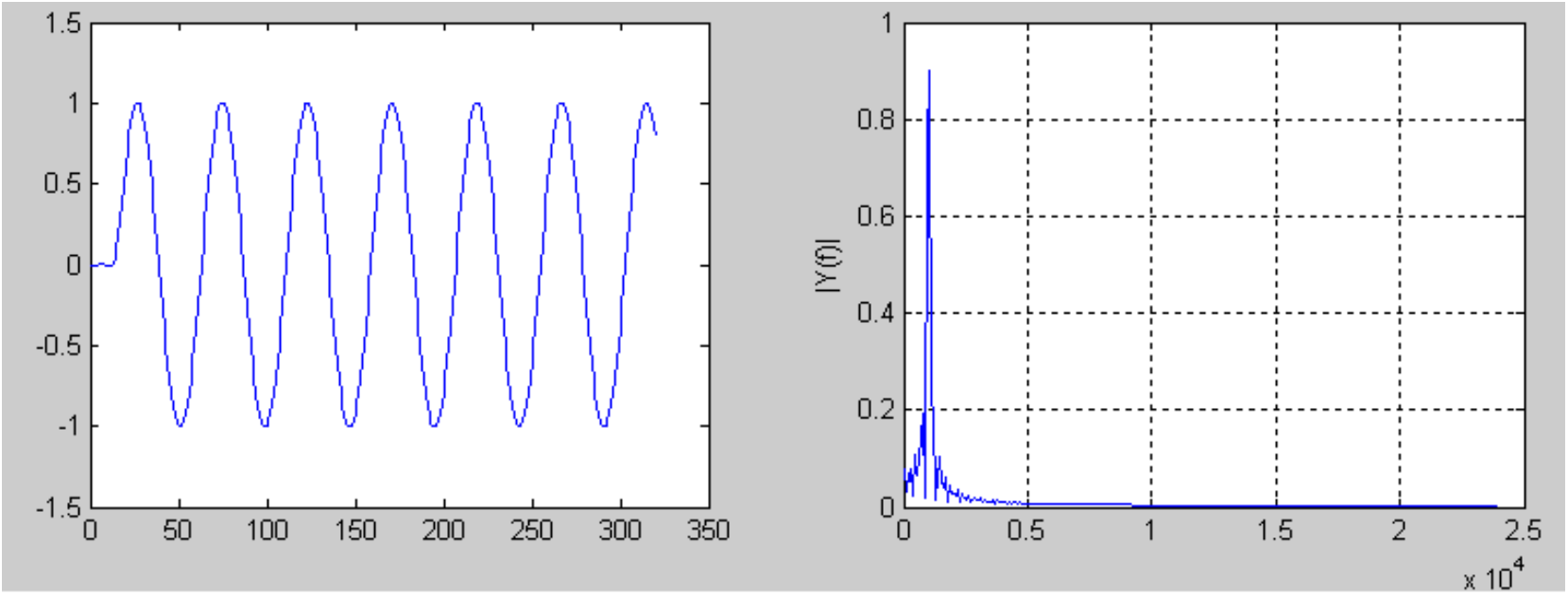
- The resulting filter coefficients and are shown below.
- Note that the filter is symmetric (a property of linear phase FIR filters) and the point of symmetry is sample 14.
- Thus the filter will have a delay of 14 samples for all frequencies.



- Input signal



- Output signal



Use CMSIS-DSP in STM32CubeIDE and example

- Check out official STM Forum (https://community.st.com/t5/stm32-mcus/configuring-dsp-libraries-on-stm32cubeide/ta-p/49637) to link/download CMSIS-DSP library in a CubeIDE project
  - The tutorial is just a reference and some modifications should be noted.
- The header files and static library of CMSIS-DSP also included in the STM32L4 firmware package
  - We can also check out the and install X-CUBE-DSPDEMO firmware package in CubeIDE (https://www.st.com/en/embedded-software/x-cube-dspdemo.html)

arm_fir_example

- Check out the STM32L4 firmware package `Drivers\CMSIS\DSP\Examples\ARM\arm_fir_example`
- To port FIR filter example to CubeIDE
  - New a projects in CubeIDE:
    - Choose B-L475E-IOT01A board
    - No need to init all peripherals
    - Enable UART1
  - Copy `math_helper.h` to Core/Inc
  - Copy `math_helper.c` and `arm_fir_data.c` to `Core/Src`,
  - Adapt and include the codes in `arm_fir_example_f32.c` into `Core/Src/main.c`

To enable printf to output message to UART1

- To enable printf to output message to UART1 (serial console), using the following function `UART_Printf_Dbg()` defined in `uart_printf.h` and `uart_printf.c` (please see the folder in our course web at https://cool.ntu.edu.tw/courses/43856/files/folder/printf_using_uart1_stm32L4 node)
  - Copy `uart_printf.c` and `uart_printf.h`, respectivey, to `Core/Src` and `Core/Inc`
  - And the following line in Core/Src/mainc.c

    `#include "uart_printf.h"`

- Alternatively, TA suggest to override the following function:

```
int __io_putchar(int ch)
{
      HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 0xFFFF);
      return ch;
}
```

CMSIS-DSP/Examples/ARM/arm_fir_example/arm_fir_example_f32.c

```
 uint32_t i;
 arm_fir_instance_f32 S;
 arm_status status;
 float32_t  *inputF32, *outputF32;

 /* Initialize input and output buffer pointers */
 inputF32 = &testInput_f32_1kHz_15kHz[0];
 outputF32 = &testOutput[0];
 /* Call FIR init function to initialize the instance
structure. */
 arm_fir_init_f32(&S, NUM_TAPS, (float32_t
*)&firCoeffs32[0], &firStateF32[0], blockSize);

 /* ----------------------------------------------------------------
-
 ** Call the FIR process function for every blockSize
samples
 ** ----------------------------------------------------------------
*/
 for(i=0; i < numBlocks; i++)
 {
   arm_fir_f32(&S, inputF32 + (i * blockSize), outputF32
+ (i * blockSize), blockSize);
  }
```

```
/* ----------------------------------------------------------------
---
 ** Compare the generated output against the
reference output computed
 ** in MATLAB.
 ** ----------------------------------------------------------------
- */

snr = arm_snr_f32(&refOutput[0], &testOutput[0],
TEST_LENGTH_SAMPLES);

status = (snr < SNR_THRESHOLD_F32) ?
ARM_MATH_TEST_FAILURE : ARM_MATH_SUCCESS;
```